

```
delegate:self  
cancelButtonTitle:@"OK"  
otherButtonTitles:nil];
```

The parameter included in the initialiser of the UIAlertView class is called the delegate. If you set this parameter to self, the current object is responsible for handling all the events fired by this instance of the UIAlertView class. If you are not going to handle events fired by this instance at all, you can simply set it to nil:

```
UIAlertView *alert = [[UIAlertView alloc]  
initWithTitle:@"Hello"  
message:@"This is an alert view"  
delegate:nil  
cancelButtonTitle:@"OK"  
otherButtonTitles:nil];
```

With multiple buttons on the alert view, we were handling the methods defined in the UIAlertViewDelegate protocol to know which button was tapped. This can be either done by implementing it in the same class in which the UIAlertView class was instantiated or by creating a new class to implement the method like:

```
//NewClass.m  
@implementation NewClass  
- (void)alertView:(UIAlertView *)alertView  
clickedButtonAtIndex:(NSInteger)buttonIndex {  
    NSLog([NSString stringWithFormat:@"%d", buttonIndex]);  
}  
@end
```

You also need to create an instance of this NewClass set it as the delegate to ensure that the alert view knows where to look for the method:

```
NewClass *myDelegate = [[NewClass alloc] init];  
UIAlertView *alert = [[UIAlertView alloc]  
initWithTitle:@"Hello"  
message:@"This is an alert view"  
delegate:myDelegate  
cancelButtonTitle:@"OK"  
otherButtonTitles:@"Option 1", @"Option 2", nil];  
[alert show]; 
```